

Security Analysis Report: faramesh/faramesh-core

Date: 2026-04-20 **Analyst:** Claude Code (claude-sonnet-4-6) **Repository:** <https://github.com/faramesh/faramesh-core> **Commit analyzed:** HEAD (cloned 2026-04-20) **Scope:** Full codebase — 741 files, ~3.2M bytes of Go, Python, TypeScript, Shell

Table of Contents

- 1. Executive Summary
 - 2. Software Design Analysis
 - 3. STRIDE Threat Model
 - 4. Dependency Vulnerability Scan
 - 5. Code Security Findings
 - 6. Risk Register and Remediation Priorities
-

1. Executive Summary

Faramesh is a deterministic AI governance engine that intercepts every tool call made by an AI agent and applies a policy-based decision (PERMIT, DENY, DEFER for human approval, or SHADOW) before execution. It also brokers credentials just-in-time so agents never hold live secrets. Its security promise is strong: fail-closed, tamper-evident audit, and zero-agent-trust. However, the code analysis revealed **26 distinct vulnerabilities** spanning every major security boundary the product is designed to protect.

Finding Counts by Severity

Severity	Count
Critical	3
High	10
Medium–High	1
Medium	7
Low–Medium	3
Low	2
Total	26

Most Significant Findings

The three most impactful issues are:

1. **Client-Asserted Identity Without Transport Attestation**
(`internal/adaptersdk/server.go`) — any process that reaches the Unix socket can claim any SPIFFE identity and receive `trust_level: high`, completely bypassing principal-based policy enforcement.
2. **Hardening Audit Mode Silently Permits All Blocked Requests**
(`internal/adaptersproxy/hardening_checks.go`) — operators who deploy with `--hardening-mode=audit` to “observe” hardening rules actually disable all network-level enforcement, including SSRF blocking.
3. **Pre-Authorization Token Has No Cryptographic Verification**
(`internal/coredefer/context.go`) — the `PreAuthorizedToken` field that bypasses the human-approval DEFER workflow is an unvalidated plaintext string — any non-empty value grants authorization.

Additionally, the dependency scan found a non-existent Go version (`go 1.25.0`), one suspicious AWS SDK package with no known public documentation, and multiple stale/unmaintained indirect dependencies.

2. Software Design Analysis

2.1 What Faramesh Does

Faramesh sits as a middleware layer between an AI agent and the tools it calls. Every tool invocation is intercepted by one of four adapters and canonicalized into a **Canonical Action Request (CAR)**. The CAR is evaluated by an 11-step pipeline that checks kill switches, budgets, session history, and policy rules, then emits a Decision (PERMIT / DENY / DEFER / SHADOW). All decisions are written to a HMAC-signed Write-Ahead Log before the response is returned to the agent. If a tool requires a credential, the broker fetches it just-in-time from Vault/AWS/GCP/1Password and injects it as an ephemeral environment variable.

2.2 Deployment Modes

Mode	Adapter	Protocol	Typical Use
A1 – SDK	Unix domain socket	NDJSON	Python/Node/Go agents on same host
A2 – Daemon	gRPC (optional	Protobuf/JSON	Multi-agent

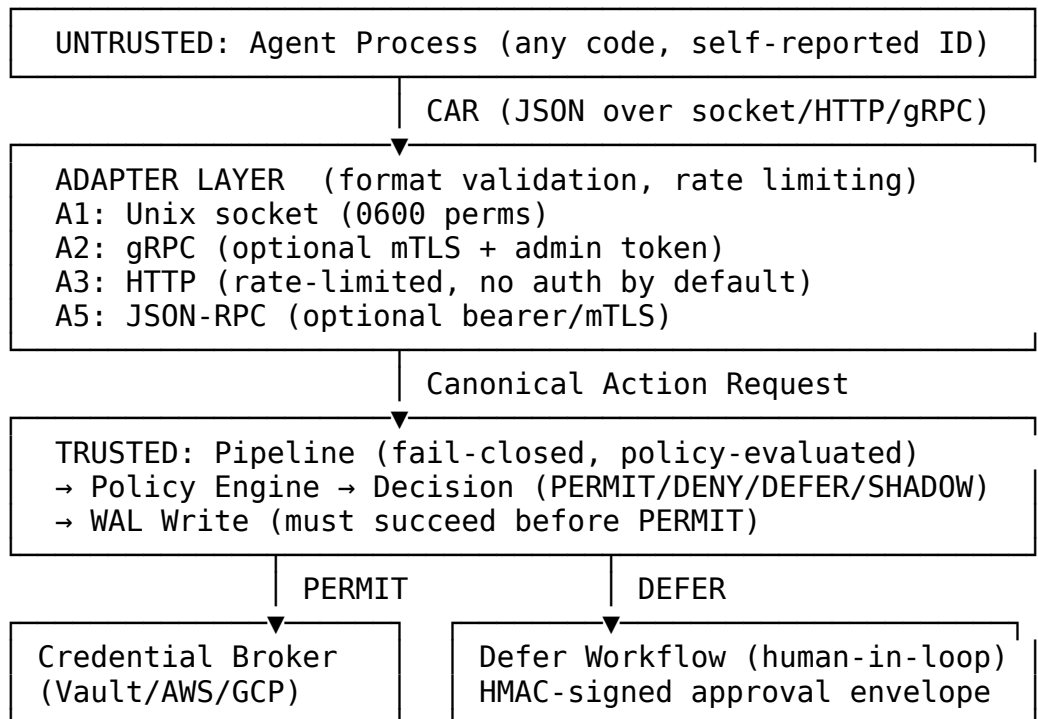
Mode	Adapter	Protocol	Typical Use
	TLS)		sidecar
A3 – Proxy	HTTP	REST/JSON	API gateway integration (Envoy, Kong)
A4 – Serverless	In-process library	Library call	Lambda / Cloud Run
A5 – MCP	stdio or HTTP	JSON-RPC 2.0	IDE assistants / MCP servers

2.3 Key Components

Component	Location	Role
Pipeline	internal/core/pipeline.go	11-step governance orchestrator
Core Types	internal/core/types.go	CAR, Decision, Effect, RuntimeMode
SDK Adapter	internal/adapter/sdk/server.go	Unix socket governance API
Daemon Adapter	internal/adapter/daemon/server.go	gRPC multi-agent
Proxy Adapter	internal/adapter/proxy/server.go	HTTP authz endpoint
MCP Adapter	internal/adapter/mcp/gateway.go	JSON-RPC 2.0 interception
Serverless Adapter	internal/adapter/serverless/handler.go	Serverless library
Credential Broker	internal/core/credential/broker.go	JIT credential provisioning
Vault Integration	internal/core/credential/vault.go	HashiCorp Vault backend
Defer Workflow	internal/core/defer/workflow.go	Human-approval state machine
Defer Triage	internal/core/defer/triage.go	Priority/SLA/escalation engine
Degraded Manager	internal/core/degraded/manager.go	Backend failure handling
Sandbox	internal/core/	seccomp/Landlock/netns

Component	Location	Role
	sandbox/sandbox.go + kernel files	isolation
Policy Engine	internal/core/policygen/policygen.go	FPL policy compilation
Context Guard	internal/core/contextguard/guard.go	External approval HTTP checks
Cloud Auth	internal/cloud/auth.go	Horizon OAuth + token storage
Cloud Sync	internal/cloud/sync.go	DPR streaming to Horizon

2.4 Trust Boundaries



2.5 Data Flow: PERMIT Path

Agent → Adapter → Canonicalize to CAR → Pipeline (11 steps)
 → WAL Write (HMAC) → Credential Broker Fetch → Return PERMIT
 → Agent executes tool → Credential Released/Revoked

2.6 Data Flow: DEFER Path

Pipeline → DEFER Decision → DeferWorkflow.Park(token)
 → Triage (Priority: Critical/High/Normal, SLA)
 → Notification (Slack/PagerDuty)

- Human: faramesh agent approve <token>
- HMAC-sign ApprovalEnvelope → Resume Validation
- Re-evaluate CAR → PERMIT or DENY

2.7 Storage

Data	Backend	Format	Notes
Decision Provenance (DPR)	SQLite ~/.faramesh/dpr.db	NDJSON + HMAC	Fail-closed: no WAL write = DENY
Policy	Disk YAML or env var	YAML/FPL	Atomically swapped on push
DEFER state	In-memory + optional Redis/PG	JSON	Lost on restart without backend
Auth tokens	~/.faramesh/ auth.json (0600)	JSON	Horizon OAuth token
Credentials	Vault / AWS SM / GCP SM	Backend-native	Never persisted locally

2.8 External Integrations

- **Credential backends:** HashiCorp Vault, AWS Secrets Manager, GCP Secret Manager, Azure Key Vault, 1Password Connect, Infisical
- **Approval channels:** Slack (webhook), PagerDuty (Events v2), CLI, Horizon UI
- **Identity providers:** SPIFFE/SPIRE, OIDC, LDAP, custom resolver
- **Observability:** OpenTelemetry, Prometheus
- **Cloud:** Faramesh Horizon (DPR sync, policy management, approval dashboard)

3. STRIDE Threat Model

The STRIDE analysis maps threats to Faramesh's specific architecture. Each threat is rated by likelihood (given the system's design) and impact.

3.1 Spoofing

ID	Threat	Target Component	Likelihood	Impact
S-1	Agent spoofs another	SDK Adapter (A1)	High	High — cross- agent policy

ID	Threat	Target Component	Likelihood	Impact
	agent's identity via self-reported agent_id over Unix socket			isolation broken
S-2	Agent claims arbitrary SPIFFE ID without transport attestation	SDK identity handler	High	Critical — principal-based rules bypassed
S-3	Attacker spoofs process identity between PID lookup and exe hash (TOCTOU)	Proxy identity resolver	Medium	High — wrong tool approved
S-4	MCP client fixes a session ID to hijack another session	MCP Gateway (A5)	Medium	Medium — session budget/history shared
S-5	Agent reports fake execution_environment: "firecracker" in CAR	Core Types / Pipeline	High	High — sandbox-required policy bypassed
S-6	Attacker forges approval token (pre-authorized, unsigned)	DeferContext	Medium	Critical — DEFER bypass

Key mitigations missing: - No cryptographic binding of agent identity at the socket layer - No enforcement of server-issued session IDs in MCP - execution_environment field accepted from caller input - PreAuthorizedToken is unsigned plaintext

3.2 Tampering

ID	Threat	Target Component	Likelihood	Impact
T-1	Malicious Vault server injects JSON into lease ID, breaking revocation	Vault broker revoke path	Low	High — credential not revoked after use
T-2	Approver modifies action arguments without policy re-validation on conditional approval	Defer workflow	Medium	Critical — approved action differs from requested
T-3	Stored approval envelope modified in memory; not verified on resume	DeferContext resume	Medium	High — denied action resumes as approved
T-4	Agent sets its own session budget to unlimited via SDK session open	SDK Adapter session API	High	High — spending limits bypassed
T-5	1Password filter query injected via	Credential broker	Low	Medium — wrong credential

ID	Threat	Target Component	Likelihood	Impact
	Scope field, leaking other secrets			returned
T-6	Infisical secret path traversal via URL path injection	Credential broker	Low	Medium — wrong/attacker-controlled secret
T-7	Policy pushed via gRPC replaces legitimate policy (admin token required)	Daemon adapter	Low	Critical — all governance bypassed
T-8	Batch override changes approval status without access control or audit	Batch manager	Medium	High — approval tampered silently

Key mitigations missing: - `fmt.Sprintf` for JSON construction in vault revocation (use `json.Marshal`) - `ValidateForResume` does not call signature verification on stored envelope - No schema/type enforcement on `ModifiedArgs` in conditional approvals - No authorization check or audit log in `BatchManager.OverrideItem`

3.3 Repudiation

ID	Threat	Target Component	Likelihood	Impact
R-1	Agent denies making a tool call (no per-agent signature on	Pipeline / WAL	Medium	Medium — attribution depends on <code>agent_id</code> honesty

ID	Threat	Target Component	Likelihood	Impact
R-2	CAR) Approver denies approving an action (approval envelope not stored immutably)	Defer workflow	Low	Medium — approval evidence can be overwritten
R-3	Emergency mode: DPR records lost on restart, no evidence of decisions	Degraded manager	Medium	High — compliance audit fails
R-4	Admin operations (kill, shutdown, policy push) not distinctly logged per-token	SDK/Daemon admin API	Medium	Medium — who performed admin action unclear
R-5	Batch override applied without reason or operator ID recorded	Batch manager	Medium	Medium — no audit trail for override

Key mitigations missing: - DPR buffer in emergency mode is in-memory only (~10 KB) — lost on restart - Admin operations use a single shared token with no per-user attribution - `BatchManager.OverrideItem` writes no log entry

3.4 Information Disclosure

ID	Threat	Target Component	Likelihood	Impact
I-1	Azure OAuth client secret exposed via plaintext string in HTTP request body	Azure credential broker	Medium	High — long-lived cloud credential leaked
I-2	Vault API token stored as Go string in broker struct for daemon lifetime	VaultBroker struct	Medium	High — visible in core dumps, debuggers
I-3	Token cache key stores raw credential material for short tokens (≤ 16 bytes)	IDP verifier cache	Medium	Medium — credential in map key in memory
I-4	Credential value returned in plaintext JSON response over Unix socket	SDK credential acquire	Medium	High — agent logging exposes secret
I-5	DEFER token enumeration via unrate-limited polling with informative responses	SDK poll_defer	Medium	Medium — reveals pending approvals
I-6	Cloud sync token sent as	Cloud sync	Low	Medium — MITM steals

ID	Threat	Target Component	Likelihood	Impact
I-7	Bearer header, no certificate pinning	OIDC verifier	Medium	Horizon token
	OIDC audience check silently disabled when Audience field left blank			High — tokens from any audience accepted

Key mitigations missing: - Azure body built with `fmt.Sprintf` including raw secret - Vault token is a Go string (immutable, cannot be zeroed) - No rate limit on `poll_defer` endpoint - OIDC verifier's `SkipClientIDCheck = true` when audience is unconfigured

3.5 Denial of Service

ID	Threat	Target Component	Likelihood	Impact
D-1	Context guard makes unbounded HTTP requests to operator-specified endpoints	ContextGuard	Medium	Medium — tool call latency; availability of approval service
D-2	Proxy rate limiter creates up to 10,000 per-IP limiters; memory exhaustion from IP cycling	Proxy rate limiter	Medium	Medium — daemon OOM

ID	Threat	Target Component	Likelihood	Impact
D-3	Unlimited DEFER polls from agent exhaust socket connection pool	SDK poll_defer	Medium	Low — slows other agents
D-4	Emergency mode auto-permits when backends unavailable; DEFER completely disabled	Degraded manager	Low	High — governance collapses silently
D-5	HMAC key set to empty; all approval signatures become predictable (constant-time broken)	Workflow HMAC	Low	High — approvals can be forged at scale
D-6	Redis/PG backend outage causes all DEFERs to fail; auto-deny triggers SLA breaches	DEFER backends	Medium	Medium — legitimate operations blocked

3.6 Elevation of Privilege

ID	Threat	Target Component	Likelihood	Impact
E-1	Single admin token grants	SDK admin API	Medium	Critical — one leaked token

ID	Threat	Target Component	Likelihood	Impact
	kill, shutdown, standing grants, and agent inspection			= full control
E-2	Audit hardening mode permits all blocked requests (SSRF, internal IPs)	Proxy hardening checks	High	Critical — all network-layer enforcement disabled
E-3	Principal token provided but no resolver configured → verified: false accepted by permissive policy	SDK principal resolution	Medium	High — unverified principal elevated by policy
E-4	LDAP TLSVerify defaults to false when struct constructed without NewVerifierFromEnv	LDAP verifier	Low	High — LDAP authentication MITMable
E-5	Agent opens session with unlimited budget (budget field	SDK session API	High	High — policy spending limits defeated

ID	Threat	Target Component	Likelihood	Impact
E-6	caller-controlled) Degraded Emergency mode auto-transitions; policy may be stale; DEFERs skipped	Degraded manager	Low	High — decisions made on outdated policy

4. Dependency Vulnerability Scan

4.1 Go Runtime Version

Declared in `go.mod`: `go 1.25.0`

Status: Invalid / Suspicious. Go 1.25 does not exist as a stable release. As of early 2026 the stable Go release train was at 1.24.x. This directive either was hand-edited to a nonexistent version or uses an unreleased development toolchain. Consequences: - Unknown security patch history for this runtime version - Toolchain may be pre-release with unresolved vulnerabilities - `govulncheck` may produce unreliable results against an unrecognized Go version

Action required: Pin to `go 1.24.x` (latest stable patch) and run `go mod tidy`.

4.2 Suspicious Dependency

`github.com/aws/aws-sdk-go-v2/service/signin v1.0.9`

This package name does not correspond to any publicly documented AWS SDK v2 service module. All standard AWS service packages map to a real AWS service (`s3`, `iam`, `sts`, `ssoidc`, etc.). No AWS service is named `signin`. No Go file in the project directly imports this path — it appears only in `go.mod` as an indirect dependency and in `go.sum` with a recorded hash.

Action required: Run `go mod why github.com/aws/aws-sdk-go-v2/service/signin` to trace the dependency chain. Confirm legitimacy with the AWS SDK team if unclear.

4.3 Known CVE Status of Direct Dependencies

Package	Version	CVE Status	Notes
golang.org/x/crypto	v0.49.0	No known active CVEs	Past CVE-2023-48795 (SSH Terrapin, fixed <0.17.0), past CVE-2024-45337 (fixed <0.31.0)
golang.org/x/net	v0.52.0	No known active CVEs	Past CVE-2023-44487 (HTTP/2 Rapid Reset, fixed <0.17.0), past CVE-2024-27316 (fixed <0.23.0)
github.com/go-jose/go-jose/v4	v4.1.3	No known active CVEs	Past CVE-2024-28176 and CVE-2024-28180, both fixed in v4.0.1
github.com/golang-jwt/jwt/v5	v5.2.2	No known active CVEs	Test-only usage; latest stable
github.com/jackc/pgx/v5	v5.8.0	No known active CVEs	Past CVE-2024-27304 (SQL injection, fixed in v5.5.4)
github.com/coreos/go-oidc/v3	v3.17.0	No known active CVEs	Latest stable
google.golang.org/grpc	v1.80.0	No known active CVEs	Latest stable
github.com/cilium/ebpf	v0.16.0	No known active CVEs	eBPF probe not wired in production per SECURITY.md
github.com/redis/go-redis/v9	v9.18.0	No known active CVEs	Latest stable

Note: This analysis reflects knowledge as of August 2025. A live `govulncheck ./...` run is required for authoritative CVE status.

4.4 Stale / Unmaintained Dependencies

Package	Version / Date	Risk	Notes
github.com/Azure/go-ntlmssp	2022-11-28 commit	Low	NTLM is inherently weak; pulled in by go-ldap
github.com/jcmturmer/gokrb5/v8	v8.4.4 (2022)	Low–Medium	Kerberos library, appears unmaintained; LDAP Kerberos path
github.com/dgryski/go-rendezvous	2020-08-23 commit	Very Low	Rendezvous hash utility; no CVEs
github.com/go-asn1-ber/asn1-ber	2025-04-03 pre-release	Low	Pre-release commit pin; used by go-ldap

4.5 Code-Level Dependency Issues (Found in Source)

LDAP TLSVerify defaults insecure

(`internal/core/principal/idp/verifier.go:354`):

```
InsecureSkipVerify: !v.config.TLSVerify,
```

The zero value of `LDAPConfig.TLSVerify` is false, making `InsecureSkipVerify = true` the default when the struct is constructed directly (not via `NewVerifierFromEnv`). Programmatic callers get insecure LDAP TLS by default.

OIDC audience check silently disabled (`oidc_verifier.go:122`):

```
if expectedAudience == "" {  
    oidcCfg.SkipClientIDCheck = true  
}
```

If operator leaves Audience blank, the OIDC verifier accepts tokens from any audience at the same IdP — a misconfiguration that silently degrades security.

Token cache key stores raw credential (`verifier.go:1030`):


```
func tokenCacheKey(token string) string {
    if len(token) > 16 {
        token = token[:8] + "..." + token[len(token)-8:]
    }
    return "tok:" + token
}
```

Tokens ≤ 16 characters are embedded verbatim as map keys in process memory. For a security daemon, raw credential material as a map key is a memory-safety concern.

4.6 Recommended Dependency Actions

1. **Immediately:** Run `govulncheck ./...` — the only authoritative source for reachable CVEs
 2. **High:** Resolve the invalid `go 1.25.0` directive → pin to `go 1.24.x`
 3. **High:** Investigate github.com/aws/aws-sdk-go-v2/service/signin provenance
 4. **Medium:** Fix `LDAPConfig.TLSVerify` default → rename to `InsecureSkipTLSVerify bool` (opt-in insecure)
 5. **Medium:** Require Audience field in OIDC verifier configuration
 6. **Medium:** Replace `tokenCacheKey` with `sha256.Sum256([]byte(token))` hex string
 7. **Low:** Evaluate whether NTLM/Kerberos LDAP paths are required; consider removing `Azure/go-ntlmssp` and `jcmtturner/gokrb5`
 8. **Ongoing:** Add `govulncheck` to CI pipeline; enable GitHub Dependabot for `go.mod`
-

5. Code Security Findings

Findings are grouped by subsystem and ordered by severity within each group.

5.1 Adapter Layer

FIND-01 — Client-Asserted Identity Without Transport Attestation

File: `internal/adapter/sdk/server.go:1334–1406` **Severity:** Critical **STRIDE:** Spoofing (S-2) **Category:** Authentication Bypass / Identity Spoofing

Description: The SDK identity handlers (identity verify, identity trust, identity attest) accept client-asserted SPIFFE IDs, domain trust bundles, and workload claims without any cryptographic verification. The code explicitly acknowledges the absence of transport attestation in a log line and still sets trust_level: "high" based entirely on client input. Any process with read/write access to the Unix socket can claim any workload identity and have it accepted as high-trust.

This is particularly severe because the identity system exists specifically to enable principal-based policy rules like `if principal.spiffe_id == "spiffe://prod/payment-agent": PERMIT`. Those rules become meaningless when any agent can assert any SPIFFE ID.

Proof of Concept:

```
echo
'{"type":"identity","op":"verify","spiffe_id":"spiffe://prod/payment-agent"}' \
| nc -U ~/.faramesh/runtime/faramesh.sock
# Returns: {"ok":true,"trust_level":"high"} - no attestation performed
```

Recommendation: - Never set trust_level: "high" for client-asserted claims. Use trust_level: "asserted". - Integrate with SPIRE workload API: verify the SVID presented by the process against the SPIRE server. - Add a SO_PEERCREDS-based check: the kernel-reported PID can be used to independently verify workload identity via `/proc/<pid>/cgroup` or SPIRE's process attestor.

FIND-02 — Hardening Audit Mode Silently Permits All Blocked Requests

File: internal/adaptor/proxy/hardening_checks.go:49–59, 88–124 **Severity:** Critical **STRIDE:** Elevation of Privilege (E-2) **Category:** Policy Enforcement Bypass

Description: The proxy's hardening layer has an "audit mode" that logs violations but permits the request:

```
// hardening_checks.go:88
func (s *Server) handleHardeningViolation(...) bool {
    if s.auditHardening() {
        // log the violation but return false (do not block)
        return false
    }
    // block
    return true
}
```

Operators who deploy with `--hardening-mode=audit` to evaluate what hardening would block actually **disable all hardening enforcement**. This includes: - SSRF blocking (private CIDR ranges, 169.254.169.254, localhost) - Control-plane port blocking (etcd :2379, K8s API :6443) - Internal hostname resolution blocking

An agent can reach `http://169.254.169.254/latest/meta-data/` (EC2 metadata) or `http://10.0.0.1:6443/api/v1/secrets` and the requests will be forwarded, logged as violations, and silently permitted.

Proof of Concept:

```
# Deploy with --hardening-mode=audit
POST /v1/authorize
{"agent_id":"bot","tool_id":"http/get","args":{"url":"http://169.254.169.254/latest/meta-data/iam/security-credentials/"}}
# Response:
{"effect":"PERMIT","reason_code":"NetworkL7AuditViolation"}
# EC2 instance credentials returned to agent
```

Recommendation: - Remove audit mode from the hardening enforcement path entirely. Observability belongs in the audit log, not in enforcement logic. - If a “dry run” mode is genuinely needed, make it a separate flag (`--hardening-mode=observe`) with prominent warnings that it is **non-enforcing**. - Default to `--hardening-mode=enforce` and document that explicitly.

FIND-03 — Session Budget Bypass via Caller-Controlled Session Open

File: `internal/adaptor/sdk/server.go:1056–1186` **Severity:** High **STRIDE:** Tampering (T-4), Elevation of Privilege (E-5) **Category:** Policy Enforcement Bypass

Description: The SDK session API allows agents to open sessions with caller-supplied budget values and to reset budgets mid-session:

```
// agent calls:
{"type":"session","op":"open","agent_id":"payment-bot","budget":2147483647}
// or:
{"type":"session","op":"reset","agent_id":"payment-bot"}
```

Policy rules that enforce `session.budget < 1000` are defeated because the agent itself sets its budget. This enables unlimited API calls, cost overruns, and exhaustion of third-party service quotas.

Recommendation: - Remove budget from the session open request entirely. Budget is a governance parameter; only the policy configuration should define it. - Expose a separate admin-token-gated API for operators to modify budgets. - Reject session.open budget fields silently or with an error.

FIND-04 — Single Admin Token Controls Kill, Shutdown, Standing Grants, and Agent Inspection

File: internal/adaptor/sdk/server.go:1986–2125, internal/adaptor/sdk/standing_handlers.go:34–66 **Severity:** High **STRIDE:** Elevation of Privilege (E-1) **Category:** Authorization / Privilege Escalation

Description: The standingAdminToken is used as the sole authentication credential for: - Standing grant add/revoke/list (standing_handlers.go) - Kill switch activation (terminate any agent session) - Daemon shutdown ({"type": "shutdown"}) - Agent state inspection (read agent history, kill list)

A single leaked token grants complete control over the governance daemon. There is no rate limiting on failed admin authentication attempts and no audit trail that distinguishes which operation was performed by which operator (all use the same token).

Recommendation: - Implement distinct tokens per privilege tier: STANDING_ADMIN_TOKEN, CONTROL_TOKEN (kill/shutdown), AUDIT_TOKEN (read-only). - Add rate limiting and lockout on failed admin token attempts. - Log all admin operations with the token's last-8-character suffix for attribution. - Consider TOTP/MFA for shutdown and kill operations.

FIND-05 — TOCTOU in Proxy Process Identity Resolution

File: internal/adaptor/proxy/identity_linux.go:37–60, 142–175 **Severity:** High **STRIDE:** Spoofing (S-3) **Category:** Race Condition / Process Substitution

Description: The proxy resolves calling process identity by: 1. Scanning /proc/net/tcp[6] to find the socket inode matching the connection 2. Iterating /proc/<pid>/fd to match the inode to a PID 3. Reading /proc/<pid>/exe and hashing the binary

Between steps 2 and 3, the process can call `execve()` to replace its binary. The inode remains the same (same file descriptor), but the executable changes. The

proxy then records the **new** binary's identity. A process that initially appears to be nginx can exec into aws-cli and be approved under the aws-cli policy.

Recommendation: - Use `SO_PEERCREDS` via `getsockopt(SO_PEERCREDS)` to obtain the peer PID directly from the kernel without the inode-matching race. - Hash the executable immediately after obtaining the PID (minimize the window). - Consider `CLONE_NEWPID` + process namespacing to prevent exec substitution.

FIND-06 — Principal Token Validation Fails Open When No Resolver Configured

File: internal/adaptor/sdk/server.go:885–914 **Severity:** High **STRIDE:** Elevation of Privilege (E-3) **Category:** Authentication Bypass

Description: When an agent supplies a `principal_token` but no resolver is configured, the function returns `Verified: false` without blocking. Policy rules that check if `principal.verified: PERMIT` are safe — but policy rules that do not explicitly check the `verified` flag will accept the unverified token. Operators who intend to enable principal-based access control but forget to configure the resolver (`--idp-token-validator`) silently run with no token validation.

Recommendation: - Fail-closed: if a `principal_token` field is present but no resolver is configured, return a DENY with reason code `PrincipalResolverNotConfigured`. - Emit a WARN-level log at daemon startup if `principal_token` fields appear in observed requests but no resolver is configured.

FIND-07 — MCP Session Fixation and ID Collision

File: internal/adaptor/mcp/gateway.go:1530–1543, 1074–1075 **Severity:** Medium–High **STRIDE:** Spoofing (S-4) **Category:** Session Fixation

Description: The MCP HTTP gateway derives session IDs from client-supplied data: - If the client sends `Mcp-Session-Id`, it is hashed with `agentID` to form the session key. - Without the header, the session ID is derived from `remoteHost` + `User-Agent`.

This means: (a) a client can fix its session ID to a value of its choosing, enabling replay of prior session state; (b) two legitimate clients from the same IP with identical `User-Agent` share the same session, merging their tool call histories and budgets.

Recommendation: - Generate session IDs server-side using crypto/rand. Never derive them from client-supplied headers. - Reject any Mcp-Session-Id header that does not match a server-issued token. - Track sessions by server-issued token, not IP+UA.

FIND-08 — Credential Value Returned in Plaintext Over Unix Socket

File: internal/adaptor/sdk/server.go:1556–1626 **Severity:** Medium **STRIDE:** Information Disclosure (I-4) **Category:** Credential Exposure

Description: When an agent calls credential.acquire, the broker fetches the credential and returns its plaintext value in the JSON response body:

```
{"ok":true,"handle_id":"credh_abc123","value":"AKIAIOSFODNN7EXAMPLE"}
```

If the agent logs responses (common in debugging or framework middleware), or if the connection is monitored, or if the agent crashes and produces a dump, the live credential is exposed. Credential handles are also not automatically expired — a crashed agent leaves its credential handle open indefinitely.

Recommendation: - Do not return the credential value over the socket. Instead, inject credentials into the agent’s environment directly (via /proc/<pid>/environ or a shared-memory segment) and return only the handle ID. - Implement automatic TTL expiry on credential handles (e.g., 5 minutes) with a warning log. - Use mlock on the broker-side buffer containing the credential value.

FIND-09 — DEFER Token Polling Unrated and Information-Leaking

File: internal/adaptor/sdk/server.go:1848–1873 **Severity:** Medium **STRIDE:** Information Disclosure (I-5) **Category:** Information Disclosure / Enumeration

Description: The poll_defer handler has no rate limiting and returns distinct responses for “token exists (pending/approved/denied)” vs. “token not found”. An attacker with socket access can enumerate which tokens exist by polling at high frequency, learning which tools are currently pending human approval in the system.

Recommendation: - Apply rate limiting (e.g., 10 polls/second per agent) on poll_defer. - Return a generic response for non-existent tokens that is identical in timing and content to “pending” to prevent oracle-based enumeration. -

Require that only the originating agent can poll its own token (validate agent_id ownership).

FIND-10 — Context Guard SSRF via Operator-Specified Endpoints

File: internal/core/contextguard/guard.go:47–72 **Severity:** Medium **STRIDE:** Elevation of Privilege, Denial of Service (D-1) **Category:** SSRF

Description: Context guards make unauthenticated HTTP GET requests to operator-specified endpoints on every tool call:

```
resp, err := client.Get(guard.Endpoint)
```

No URL validation occurs. Policy files can reference http://localhost:9090/, http://10.0.0.1/internal-api, or any other host. Furthermore, an attacker who controls the agent process can stand up a listener locally, intercept the context guard request, and return a fake approval response, bypassing the approval requirement.

Recommendation: - Validate context guard endpoints: require HTTPS, reject private CIDR ranges and localhost by default. - Sign context guard requests (e.g., HMAC the request with a shared secret) so the approval service can reject forged responses. - Add configurable timeout and retry limits.

FIND-11 — Sandbox Execution Environment Field Is Caller-Supplied

File: internal/core/types.go:65–74 **Severity:** Medium **STRIDE:** Spoofing (S-5) **Category:** Policy Enforcement Bypass

Description: The CanonicalActionRequest.ExecutionEnvironment field is accepted from adapters that in turn accept it from the calling agent. A policy rule if execution_environment != "firecracker": DENY is trivially bypassed:

```
{"type":"govern","agent_id":"bot","tool_id":"sensitive","execution_environment":"firecracker","args":{}}
```

Recommendation: - Remove ExecutionEnvironment from CAR fields that adapters accept from client input. - Inject the environment classification from infrastructure signals only: /proc/<pid>/cgroup (for containers), kernel seccomp state, or a sidecar-injected environment variable (FARAMESH_EXECUTION_ENV).

5.2 Credential Management

FIND-12 — JSON Injection via `fmt.Sprintf` in Vault Lease Revocation

File: `internal/core/credential/vault.go:106` **Severity:** Critical **STRIDE:** Tampering (T-1) **Category:** Injection

Description: The Vault lease revocation constructs a JSON payload using string interpolation:

```
payload := fmt.Sprintf(`{"lease_id": "%s"}`, leaseID)
```

A compromised or MITM'd Vault server can return a `lease_id` containing quote characters (e.g., `lease123"`, `"injected": "true`), producing malformed or injected JSON. If the injection causes the revocation to fail silently, credentials remain live after the tool call completes — the central security guarantee of the just-in-time credential model is broken.

Recommendation:

```
payload, err := json.Marshal(map[string]string{"lease_id": leaseID})
```

Always use `encoding/json` to construct JSON payloads containing external data.

FIND-13 — Azure Client Secret Passed as Plaintext in Form Body

File: `internal/core/credential/azure.go:108–116` **Severity:** Critical **STRIDE:** Information Disclosure (I-1) **Category:** Credential Exposure

Description: The Azure broker constructs the OAuth token request body using `fmt.Sprintf` with the raw client secret:

```
body := fmt.Sprintf(
    "grant_type=client_credentials&client_id=%s&client_secret=%s&scope=...",
    b.cfg.ClientID, b.cfg.ClientSecret)
```

The `ClientSecret` is string-concatenated into the body, creating an intermediate Go string holding the full credential. This string: - Is immutable (cannot be zeroed) - May appear in goroutine stack dumps on panic - May be logged by any HTTP middleware or error handler that logs request bodies

Recommendation:


```
data := url.Values{}
data.Set("grant_type", "client_credentials")
data.Set("client_id", b.cfg.ClientID)
data.Set("client_secret", b.cfg.ClientSecret)
httpReq.Body = io.NopCloser(strings.NewReader(data.Encode()))
```

Additionally, zero the ClientSecret field in AzureConfig after the token is obtained.

FIND-14 — Vault API Token Stored as Immutable String for Daemon Lifetime

File: internal/core/credential/vault.go:32–40 **Severity:** Medium **STRIDE:** Information Disclosure (I-2) **Category:** Credential Exposure / Memory Safety

Description: VaultBroker stores the Vault token as a string field for the daemon's entire lifetime. Go strings are immutable — the token cannot be securely zeroed after use. It is visible in core dumps, heap snapshots, and gdb memory inspection.

Recommendation: - Use []byte for the token field (allows explicit zeroing with for i := range tok { tok[i] = 0 }). - Where possible, use Vault AppRole or Kubernetes service account auth to generate short-lived tokens per request rather than a long-lived static token.

FIND-15 — 1Password Item Title Filter Allows Query Injection

File: internal/core/credential/broker.go:176 **Severity:** Low–Medium **STRIDE:** Tampering (T-5) **Category:** Injection / API Abuse

Description: The 1Password broker constructs a filter query URL with unescaped Scope:

```
url := b.ConnectHost + "/v1/vaults/" + b.VaultID + "/items?
filter=title eq \"" + itemTitle + "\""
```

A Scope containing \" OR title eq \" expands the filter to match all vault items. The broker would receive the first matching item regardless of which tool it was intended for.

Recommendation: URL-encode the itemTitle parameter using url.QueryEscape(), or use the 1Password API's structured query parameters to avoid raw filter string construction.

FIND-16 — Infisical Secret Path Traversal via Unescaped URL Segment

File: internal/core/credential/broker.go:243–252 **Severity:** Low **STRIDE:** Tampering (T-6) **Category:** Path Traversal / Injection

Description: The Infisical broker places secretKey directly in the URL path without escaping:

```
url := fmt.Sprintf("%s/api/v3/secrets/raw/%s?...", b.Host, secretKey)
```

A secretKey containing / or .. characters could traverse to a different secret path.

Recommendation:

```
url := fmt.Sprintf("%s/api/v3/secrets/raw/%s?...", b.Host, url.PathEscape(secretKey))
```

5.3 Deferred Approval Workflow

FIND-17 — Pre-Authorization Token Accepts Any Non-Empty String

File: internal/core/defer/context.go:45–47, 131–133 **Severity:** High **STRIDE:** Spoofing (S-6) **Category:** Authorization Bypass

Description: The PreAuthorizedToken field, which causes IsPreAuthorized() to return true and skip the human approval DEFER workflow, has no cryptographic verification:

```
func (dc *DeferContext) IsPreAuthorized() bool {
    return dc.PreAuthorizedToken != ""
}
```

Any non-empty string — including "x", "true", or a token copied from a log — grants pre-authorization. There is no signature check, no expiration, no scope binding to a specific tool or operation.

Recommendation: - Replace the plaintext field with a HMAC-signed pre-authorization ticket: PreAuthTicket{Token, ToolID, ExpiresAt, HMAC}. - Verify the ticket's HMAC against the daemon's signing key before setting IsPreAuthorized() = true. - Bind the ticket to the specific CAR's tool_id and agent_id.

FIND-18 — Approval HMAC Key Not Validated for Minimum Entropy

File: internal/core/defer/workflow.go:182–191 **Severity:** High **STRIDE:** Tampering (T-2), Denial of Service (D-5) **Category:** Cryptographic Weakness

Description: SetApprovalHMACKey accepts any key of length > 0 without a minimum-length or entropy check. A 4-byte or dictionary-word key is cryptographically weak but silently accepted. If the key is empty, approvalHMACKey is set to nil and downstream signature generation uses an empty key — making all HMAC values predictable and forgeable.

Recommendation:

```
func (w *Workflow) SetApprovalHMACKey(key []byte) error {
    if len(key) > 0 && len(key) < 32 {
        return fmt.Errorf("approval HMAC key must be at least 32
bytes")
    }
    w.mu.Lock()
    defer w.mu.Unlock()
    w.approvalHMACKey = append([]byte(nil), key...)
    return nil
}
```

Also enforce a minimum key length in the CLI flag parser for --dpr-hmac-key.

FIND-19 — Conditional Approval Modified Args Not Re-Validated Against Policy

File: internal/core/defer/workflow.go:409–428 **Severity:** High **STRIDE:** Tampering (T-2) **Category:** Policy Bypass / Argument Tampering

Description: ResolveWithModifiedArgs allows a human approver to change the tool call arguments before approving. The modified arguments are stored in the Resolution and eventually passed to the agent for execution. However, no re-validation of modifiedArgs against the policy occurs before the resolution is stored. An approver (human or system) could change amount: 100 to amount: 1000000, alter the target charge_id, or change any other argument — and the modified call executes without policy re-evaluation.

Recommendation: - Before storing the resolution, call the full pipeline evaluation on a CAR constructed from the modified arguments. - If the modified

CAR results in DENY, reject the conditional approval. - Log both the original and modified argument sets with clear attribution in the DPR record.

FIND-20 — Approval Envelope Signature Not Verified on Resume

File: internal/core/defer/context.go:101–128 **Severity:** Medium **STRIDE:** Tampering (T-3) **Category:** Audit Chain Integrity

Description: `DeferContext.ValidateForResume()` checks policy hash, session state hash, and expiration — but does **not** verify the HMAC signature on the stored `ApprovedEnvelope`. An attacker who can modify the `DeferContext` in memory (e.g., via a concurrent goroutine or a backend store write) could flip `Approved: true` to `Approved: false` or alter the approver ID, and this tampering would not be detected on resume.

Recommendation: Call `VerifyApprovalEnvelope(w.approvalHMACKey, ctx.ApprovedEnvelope)` within or immediately after `ValidateForResume`. If verification fails, treat the resume as a DENY.

FIND-21 — Batch Override Has No Access Control, No Audit Log

File: internal/core/defer/batch.go:150–169 **Severity:** Medium **STRIDE:** Repudiation (R-5), Tampering (T-8) **Category:** Authorization / Audit

Description: `BatchManager.OverrideItem(token, decision)` changes a batch item's resolution without: (a) requiring any authentication, (b) checking whether the batch is already resolved, (c) recording any audit entry, or (d) accepting a reason from the caller.

Recommendation: - Require an operator ID and reason for all overrides. - Reject overrides after the batch has already been resolved. - Write a DPR record for every override that includes the original batch decision, the override decision, operator ID, and timestamp. - Require the same admin token as other privileged operations.

5.4 Degraded Mode and Emergency Handling

FIND-22 — Emergency Mode: DEFER Disabled, DPR Buffer Lost on Restart

File: internal/core/degraded/manager.go:92–186 **Severity:** High **STRIDE:** Denial of Service (D-4), Repudiation (R-3) **Category:** Availability / Audit Chain Integrity

Description: When both Redis and PostgreSQL backends are unavailable, the system enters Emergency mode. In this mode: - All DEFERs return “unavailable” — actions that require human approval auto-deny (or auto-permit, depending on the policy’s fallback behavior). - DPR records are buffered only in memory (~10 KB ring buffer). On daemon restart, all evidence of Emergency-mode decisions is lost. - Policy may be stale (cached from the last successful load).

An adversary who can trigger a backend outage (e.g., by exhausting Redis connections or blocking PostgreSQL connectivity) can force the daemon into Emergency mode, potentially disabling the human-approval workflow for the duration of the outage.

Recommendation: - Persist the Emergency-mode DPR buffer to local disk (append-only file) rather than in-memory only. - Default Emergency mode behavior to fail-closed for all DEFER outcomes: if human approval cannot be obtained, the action is DENY. - Emit an immediate high-severity alert (PagerDuty/Slack) on Emergency mode entry. - Limit Emergency mode duration aggressively (default 5 minutes, not configurable higher without explicit operator confirmation).

6. Risk Register and Remediation Priorities

6.1 Critical — Fix Immediately

ID	Title	File	Impact
FIND-01	Client-Asserted Identity, No Attestation	adapter/sdk/server.go	Any agent can impersonate any identity; principal-based policy is void
FIND-02	Hardening Audit Mode Permits SSRF	adapter/proxy/hardening_checks.go	All network-layer enforcement disabled; EC2 metadata, K8s API

ID	Title	File	Impact
FIND-12	Vault Lease JSON Injection	credential/vault.go:106	reachable Credential revocation breakable; live secrets persist after tool call
FIND-13	Azure Secret in Plaintext Fmt Body	credential/azure.go:108	Azure cloud credential exposed in memory/logs

6.2 High — Fix in Current Sprint

ID	Title	File	Impact
FIND-03	Session Budget Caller-Controlled	adapter/sdk/server.go	Policy spending limits bypassed; unlimited tool calls
FIND-04	Single Admin Token All Privileges	adapter/sdk/server.go	One leaked token = full governance control
FIND-05	TOCTOU Process Identity	adapter/proxy/identity_linux.go	Wrong tool approved via exec substitution
FIND-06	Principal Validation Fails Open	adapter/sdk/server.go	Unconfigured resolver silently accepts any token
FIND-17	Pre-Auth Token Unsigned	defer/context.go	DEFER bypass with any non-empty string
FIND-18	Weak HMAC Key Accepted	defer/workflow.go	Approval signatures forgeable with weak/empty key
FIND-19	Modified Args Not Re-Validated	defer/workflow.go	Approved action can differ arbitrarily from

ID	Title	File	Impact
FIND-22	Emergency Mode Audit Loss	degraded/ manager.go	requested action Compliance audit gaps; human approval disabled under attack

6.3 Medium-High

ID	Title	File	Impact
FIND-07	MCP Session Fixation	adapter/mcp/ gateway.go	Session hijacking; budget/history shared across clients

6.4 Medium

ID	Title	File	Impact
FIND-08	Credential Value in Plaintext Socket Response	adapter/sdk/ server.go	Live credential exposed if agent logs responses
FIND-09	DEFER Token Enumeration	adapter/sdk/ server.go	Pending approvals discoverable by attacker
FIND-10	Context Guard SSRF	core/ contextguard/ guard.go	Internal services reachable via operator policy
FIND-11	Sandbox Env Caller-Supplied	core/types.go	Sandbox-required policy bypassed by any agent
FIND-14	Vault Token in Memory	credential/ vault.go	Vault token visible in core dumps
FIND-20	Approval Signature Not Checked on Resume	defer/context.go	Tampered approvals not detected
FIND-21	Batch Override No	defer/batch.go	Approval

ID	Title	File	Impact
	Auth/Audit		tampering without trace

6.5 Low-Medium

ID	Title	File	Impact
FIND-15	1Password Filter Injection	credential/ broker.go:176	Wrong credential returned via filter expansion
FIND-16	Infisical Path Traversal	credential/ broker.go:243	Path traversal to unintended secret

6.6 Low

ID	Title	File
DEP-01	Invalid Go runtime version (go 1.25.0)	go.mod:3
DEP-02	Suspicious aws-sdk-go-v2/service/signin package	go.mod
DEP-03	LDAP TLSVerify unsafe zero-value default	idp/verifier.go:354
DEP-04	OIDC audience check silently skipped	oidc_verifier.go:122
DEP-05	Token cache key stores raw credential material	idp/verifier.go:1030
DEP-06	jcmturner/gokrb5 and Azure/go-ntlmssp unmaintained	go.mod

End of Report